

4.1 Introduction and Problem Definition

Over the last few lectures we've discussed sorting. In this lecture we will discuss a related problem with some surprisingly efficient algorithms: median-finding, or more generally, selection.

The median problem is the following: given an unsorted array, find and return the median element. In other words, given an array of length n , find and return the $(n/2)$ nd smallest element. The selection problem is only slightly more general: given an array of length n and a value $k \leq n$, find and return the k th smallest element. From now on we'll mostly talk about selection.

It is obvious that selection can be done in time $O(n \log n)$: we can sort the array (using, e.g., mergesort), and then return the k th smallest element. Can we do any better?

It turns out that the answer is yes! We can do selection in $O(n)$ time.

There are a few easy cases, which we can do to warm up. For example, suppose $k = 1$. Then we are trying to find the smallest element, which we can do by simply scanning the array in $O(n)$ time and keeping track of the smallest. Similarly, if $k = n$ then a simple scan also suffices. In general, this strategy works whenever $k = O(1)$ or $k = n - O(1)$, since we can just keep track of the k smallest/largest elements we see while we do a scan.

This doesn't work for $k = n/2$, though. If we kept track of the k smallest elements, then when considering a new element in the scan we would have to figure out its place in the smallest k , which takes time $\Theta(\log k) = \Theta(\log n)$ (upper bound via binary search, lower bound something we'll see next week). So the total time would be $\Theta(n \log k) = \Theta(n \log n)$.

4.2 Quickselect

Our starting point will be *Quickselect*: the obvious variant of Quicksort for selection (rather than sorting). The key realization is that since we only care about selection, we only need to recurse on *one* side, rather than both. Slightly more formally, suppose we are given an array A of length n and an integer $k \leq n$. Then Quickselect does the following:

1. If $n = 1$, return the element.
2. Pick a pivot element p (somehow).
3. Compare each element of A to p , creating subarrays L of elements less than p and G of elements greater than p .
4.
 - (a) If $|L| = k - 1$ then return p .
 - (b) If $|L| > k - 1$ then return Quickselect(L, k).

(c) If $|L| < k - 1$ then return $\text{Quickselect}(G, k - |L| - 1)$.

It's easy to prove correctness by arguing inductively that on every call to $\text{Quickselect}(X, a)$, the original element we were looking for (the k 'th smallest of A) must be the a 'th smallest of X (do at home!).

Bounding the running time is trickier. It turns out that the main thing that matters is how “central” the pivot p is. Suppose that we get lucky, and p is exactly the $n/2$ smallest element (i.e., the pivot splits the array exactly in half). This means that after $O(\log n)$ iterations we will find the element we are looking for. This might seem like it would give a bound of $n \log n$, but in each iteration the number of comparisons we make also goes down by a factor of (approximately) 2, and thus the total number of comparisons is only $O(n)$. Slightly more formally, *if* the pivot *always* splits the array in half, then our running time is $T(n) = T(n/2) + cn$. This recurrence relation solves to $T(n) = \Theta(n)$.

On the other hand, suppose that the pivot is extreme, i.e., is either the largest or smallest element. Then we recurse on an array of size $n - 1$, so we get the recurrence relation $T(n) = T(n - 1) + cn$, which solves to $\Theta(n^2)$. This means that if we pick our pivot according to any “simple” rule, there is some input where Quickselect will take $\Theta(n^2)$ time: whatever the array is which causes the pivot to always be the largest or smallest.

4.3 Linear-time Algorithm

What if we want a linear-time algorithm? Somewhat amazingly, this turns out to be possible, even with Quickselect ! The basic idea is to deterministically find a pivot that will result in a more-or-less even split, and use Quickselect with this pivot. At first glance this seems pointless – in order to find a pivot which results in a good split we need to find one near the center, but finding an element at a specific place in the order is exactly what we're trying to solve! And it's true that we won't be able to quickly find the element that exactly splits the array, i.e., the median. But we will be able to find an element that is *close* to the median.

We will use a trick known as “median-of-medians”. This is the key to the whole algorithm. We start by splitting the array A into $n/5$ groups of 5 (we do this arbitrarily, say by just grouping consecutive subarrays of length 5 in the unsorted input A). We then compute the median of each group, and then the median of the medians, and let this be our pivot p .

Let's start by proving that p , the “median-of-medians”, is a good pivot: both L and G are reasonably small.

Lemma 4.3.1 $|L|$ and $|G|$ are both at most $7n/10$ (when the pivot p is the median-of-medians).

Proof: Let B be a group, and suppose that the median m of B is less than p . Then clearly m and the two elements less than it in B are all less than p , and thus in L . On the other hand, suppose that m is larger than p . Then m and the two elements larger than it in B are all in G .

Now note that exactly half of the groups (i.e. $n/10$ groups) have their median less than p , and half have their median larger than p (since p is the median-of-medians). Hence $|L| \geq \frac{n}{10} \cdot 3 = 3n/10$, and similarly $|G| \geq 3n/10$. Since every element other than p is in either L or G , this clearly implies

that $|L|$ and $|G|$ are both at most $7n/10$. ■

So now we have a plan: compute the median-of-medians, and use that as the pivot. But computing the median is exactly what we're trying to solve in the first place! Fortunately the size of the input has shrunk, so we can do this recursively. Putting it all together, we get the following algorithm, which I'll call BFPRT (since it was invented by Blum-Floyd-Pratt-Rivest-Tarjan). As before, we are given an array A of size n and an integer $k \leq n$.

BFPRT(A, k):

1. If $n \leq 5$, sort A and return the k th smallest element.
2. Group A into $n/5$ groups of 5, and let A' be an array of size $n/5$ containing the median of each group.
3. Let $p = \text{BFPRT}(A', n/10)$, i.e., recursively find the median p of A' (the median-of-the-medians).
4. Split A using p as a pivot into L and G .
5. Recurse on the appropriate piece:
 - (a) if $|L| = k - 1$ then return p .
 - (b) if $|L| > k - 1$ then return $\text{BFPRT}(L, k)$.
 - (c) if $|L| < k - 1$ then return $\text{BFPRT}(G, k - |L| - 1)$.

Let $T(n)$ be the running time on an array of size n (where as before we just use the worst case over possible k 's). Then step 1 takes time $O(n)$, step 2 takes time $T(n/5)$, step 3 takes time $O(n)$, and step 4 takes time at most $T(7n/10)$ (by Lemma 4.3.1). So the total running time is

$$T(n) \leq T(7n/10) + T(n/5) + cn.$$

It's a good exercise to draw out the recursion tree to see what's going on, but we can also solve by guess-and-check. Then we will guess that $T(n) \leq 10cn$. When we check this, we get that

$$T(n) \leq 10c(7n/10) + 10c(n/5) + cn = 9cn + cn = 10cn.$$

4.4 $O(n \log n)$ time Quicksort

We can now use BFPRT to get a version of Quicksort which only uses $O(n \log n)$ comparisons in the worst case (recall that traditional Quicksort uses $\Theta(n^2)$ comparisons in the worst case). The algorithm is simple: when deciding on a pivot, use BFPRT to find the median, and then use that as a pivot. Clearly this splits the input in half, so the total number of comparisons is

$$T(n) = 2T(n/2) + cn = O(n \log n),$$

where the cn term is the number of comparisons used for BFPRT plus the number used to split the array on the pivot.